

Unit 1

Topic 1: Database System Applications

Database System Applications ka matlab hai **real-life me database ka use kaha-kaha hota hai**. Aaj ke digital world me almost har system database par depend karta hai. Database systems large amount of data ko **store, manage aur retrieve** karne me help karte hain. Ye data ko organized form me rakhte hain jisse fast access possible hota hai. Different industries me database ka use alag-alag purpose ke liye hota hai. Banking sector me database ka use **customer accounts, transactions aur balance maintain karne** ke liye hota hai. Railway aur airline reservation systems me database seat availability aur booking manage karta hai. Education sector me student records, marks aur attendance store ki jati hai. E-commerce websites jaise Amazon me products, orders aur customer data database me store hota hai. Healthcare systems me patient records aur reports database ke through manage hote hain.

Database systems ki wajah se data redundancy kam hoti hai aur data consistency maintain hoti hai.

Multiple users ek hi data ko access kar sakte hain bina conflict ke. Security bhi maintain hoti hai kyunki access control apply kiya ja sakta hai. Backup aur recovery features data loss se protect karte hain.

Agar database na ho to large data ko manually manage karna mushkil ho jata hai. Isliye database systems modern applications ka backbone hote hain.

Example:

Bank me jab aap ATM se paise nikalte ho, to database turant aapka balance update karta hai.

E-commerce site par order place karte hi product stock automatically update ho jata hai.

Topic 2: Purpose of Database Systems

Database System ka main purpose hai **data ko efficiently store, manage aur retrieve karna**. Purane time me data file systems me store hota tha jisme kaafi problems hoti thi jaise data duplication, inconsistency aur difficult access. Database systems in problems ko solve karne ke liye design kiye gaye hain.

Sabse important purpose hai **data redundancy ko reduce karna**. Matlab same data multiple jagah repeat na ho. Isse storage bachti hai aur errors kam hote hain. Dusra purpose hai **data consistency maintain karna**, yani agar data ek jagah change ho to har jagah updated rahe.

Database systems **data sharing allow karte hain**, jisse multiple users ek hi data ko access kar sakte hain safely. Ye concurrency control provide karta hai taaki conflicts na ho. Security bhi ek important purpose hai, jisme unauthorized users ko data access se roka jata hai.

Database systems me **backup aur recovery features** hote hain jisse system failure ke baad data wapas restore kiya ja sakta hai. Ye data integrity maintain karta hai yani data correct aur reliable rahta hai.

Overall, database system ka goal hai data ko **organized, secure aur easily accessible banana**.

Example:

Agar ek student ka address change hota hai, to database me update karte hi sabhi systems me updated address show hoga.

Banking system me multiple users ek hi time par transaction kar sakte hain bina data conflict ke.

Topic 3: View of Data

View of Data ka matlab hai **user database ko kaise dekhta hai aur use karta hai**. Database internally complex hota hai, but users ko simple form me data dikhaya jata hai. Is concept ka goal hai complexity ko hide karna aur easy access dena. Different users ke liye different views ho sakte hain.

Database me 3 main levels hote hain:

1. **Physical Level** -> Data actually kaise store hota hai (disk, storage details)
2. **Logical Level** -> Data ka structure kya hai (tables, relations)
3. **View Level** -> User ko kya dikhana hai (custom view)

View level sabse important hai kyunki user sirf relevant data hi dekhta hai. Isse security bhi improve hoti hai kyunki unnecessary data hide hota hai. Har user ka alag role hota hai, isliye usko alag data dikhaya jata hai.

View concept se database easy aur user-friendly ban jata hai. Users ko internal working samajhne ki zarurat nahi hoti.

Example:

Bank me customer sirf apna account balance aur transactions dekh sakta hai, pura database nahi.

College me student sirf apne marks dekh sakta hai, jabki admin sab students ka data access kar sakta hai.

Topic 4: Data Abstraction (Instance and Schema)

Data Abstraction ka matlab hai **database ki complexity ko hide karke user ko simple view dena**. User ko ye nahi dekhna hota ki data internally kaise store hai, bas usko required data easily milna chahiye. Ye concept database ko easy aur efficient banata hai. Data abstraction 3 levels par kaam karta hai: physical, logical aur view level.

Is topic me do important terms hain: **Instance aur Schema**.

Instance ka matlab hai database ka **current data** kisi particular time par. Ye frequently change hota rehta hai.

Schema ka matlab hai database ka **structure ya design**. Ye batata hai ki tables kaise banenge, attributes kya honge aur relationships kaise honge. Schema rarely change hota hai.

Simple words me:

Schema = Blueprint (design)

Instance = Actual data (current state)

Data abstraction se system flexible ban jata hai. Agar schema change ho bhi jaye to user ke view par direct effect nahi padta. Ye data independence provide karta hai.

Database designers schema design karte hain aur users instance ke saath interact karte hain. Ye separation system ko maintain karna easy banata hai.

Example:

Student table ka schema ho sakta hai: (RollNo, Name, Marks).

Agar kisi time par data hai (101, Rahul, 85), to ye instance hai.

Topic 5: Data Models

Data Model ek **logical structure** hota hai jo define karta hai ki data database me **kaise organize, store aur represent** hoga. Ye database design ka base hota hai. Data model batata hai tables, relationships aur constraints kaise honge. Isse database ko samajhna aur design karna easy ho jata hai.

Different types ke data models hote hain. **Hierarchical model** me data tree structure me hota hai. **Network model** me multiple relationships allow hote hain. **Relational model** sabse popular hai jisme data tables (rows aur columns) me store hota hai. **Object-oriented model** me data objects ke form me represent hota hai.

Data model ka main goal hai **real world entities ko database structure me convert karna**. Ye data consistency aur integrity maintain karne me help karta hai. Proper data model se database efficient aur scalable banta hai. Agar model galat design ho to future me problems aa sakti hain.

Database designers pehle data model banate hain, fir uske basis par schema create hota hai. Ye development process ka important step hai.

Example:

Relational model me Student table ho sakta hai jisme columns ho: RollNo, Name, Marks.

Hierarchical model me company structure (Manager -> Employee) tree form me represent hota hai.

Topic 6: ER Model

ER Model (Entity-Relationship Model) ek **high-level data model** hai jo database ko design karne ke liye use hota hai. Ye real-world objects ko **entities aur relationships** ke form me represent karta hai. ER model graphical representation deta hai jisse database structure samajhna easy ho jata hai. Ye database design ka initial step hota hai.

Isme main components hote hain: **Entity, Attribute aur Relationship**. Entity koi real-world object hota hai (jaise Student, Teacher). Attributes uski properties hoti hain (jaise Name, Age). Relationship entities ke beech connection dikhata hai (jaise Student enrolls in Course).

ER model ka use karke designer easily identify kar sakta hai ki data kaise organize hoga. Ye redundancy reduce karta hai aur data consistency improve karta hai. ER diagrams ke through visual design banaya jata hai jo implementation se pehle planning me help karta hai.

Is model se complex systems ko simple way me represent kiya ja sakta hai. Ye relational database design ka base hota hai.

Example:

Student entity ke attributes ho sakte hain RollNo, Name.

Student aur Course ke beech relationship ho sakta hai "enrolls".

Topic 7: Relational Model

Relational Model ek **most popular data model** hai jisme data ko **tables (relations)** ke form me store kiya jata hai. Har table rows (tuples) aur columns (attributes) se milkar banti hai. Ye simple aur easy to use model hai. Isme tables ke beech relationships keys ke through define hote hain.

Har table ka ek **primary key** hota hai jo uniquely har record ko identify karta hai. Tables ke beech connection banane ke liye **foreign key** use hoti hai. Ye model data consistency aur integrity maintain karta hai. SQL (Structured Query Language) isi model par based hai.

Relational model ka main advantage hai ki ye flexible aur scalable hai. Data ko easily insert, update aur delete kiya ja sakta hai. Ye redundancy reduce karta hai aur normalization support karta hai. Most modern databases jaise MySQL, Oracle isi model ko follow karte hain.

Is model me data tabular form me hone ki wajah se understanding aur implementation dono easy hote hain.

Example:

Student table: (RollNo, Name, Marks)

Course table: (CourseID, CourseName)

Foreign key ke through Student aur Course connect ho sakte hain.

Topic 8: Database Languages DDL

DDL (Data Definition Language) database language ka part hai jo **database structure ko define aur modify** karne ke liye use hota hai. Iska use tables, schema aur constraints create karne ke liye hota hai. DDL database ka **design level control** karta hai. Ye data ko directly handle nahi karta, sirf structure manage karta hai.

Common DDL commands hain:

* **CREATE** -> new table ya database banane ke liye

* **ALTER** -> existing table me changes karne ke liye

* **DROP** -> table ya database delete karne ke liye

* **TRUNCATE** -> table ka data remove karne ke liye

DDL commands automatically commit ho jati hain, yani changes permanent ho jate hain. Ye database design ke starting phase me use hota hai. Proper DDL use se database structured aur organized banta hai. DDL constraints bhi define karta hai jaise primary key, not null, unique etc., jo data integrity maintain karte hain.

Example:

```
CREATE TABLE Student (RollNo INT, Name VARCHAR(50));
```

Is command se ek new table create hoti hai.

Topic 9: Database Languages DML

DML (Data Manipulation Language) ka use **database ke andar data ko insert, update, delete aur retrieve** karne ke liye hota hai. Ye directly data par kaam karta hai, structure par nahi. DML everyday operations ke liye sabse zyada use hota hai. Users aur applications isi ke through database se interact karte hain.

Common DML commands hain:

- * **INSERT** -> new data add karna
- * **UPDATE** -> existing data change karna
- * **DELETE** -> data remove karna
- * **SELECT** -> data retrieve karna

DML do types ka hota hai: procedural aur non-procedural. SQL mostly non-procedural DML use karta hai jisme user sirf "kya chahiye" batata hai, "kaise" system decide karta hai.

DML transactions ke andar use hota hai, isliye changes rollback bhi kiye ja sakte hain. Ye flexibility aur safety provide karta hai. Proper DML use se data accurately manage hota hai.

Example:

```
INSERT INTO Student VALUES (101, 'Rahul');
```

```
SELECT * FROM Student;
```

Ye commands data insert aur retrieve karte hain.

Topic 10: Database Languages DCL

DCL (Data Control Language) ka use **database me access control aur permissions manage karne** ke liye hota hai. Ye decide karta hai ki kaun user kya access kar sakta hai. Iska main focus **security** par hota hai. DCL unauthorized access ko rokta hai aur data protection ensure karta hai.

DCL me mainly 2 commands use hoti hain:

- * **GRANT** -> kisi user ko permission dene ke liye
- * **REVOKE** -> kisi user ki permission wapas lene ke liye

Iske through admin define karta hai ki kaun user read (SELECT), write (INSERT), update ya delete kar sakta hai. Ye multi-user environment me bahut important hota hai.

DCL se sensitive data secure rehta hai. Ye database system me **role-based access control** implement karne me help karta hai. Har user ko sirf utni hi permission milti hai jitni usko zarurat hai.

Example:

```
GRANT SELECT ON Student TO user1; -> user1 sirf data dekh sakta hai
```

```
REVOKE INSERT ON Student FROM user1; -> user1 data add nahi kar sakta
```

Topic 11: Database Languages TCL

TCL (Transaction Control Language) ka use **database transactions ko manage karne** ke liye hota hai. Transaction ka matlab hota hai operations ka ek group jo ya to **poora execute hoga ya bilkul nahi hoga**. TCL database me consistency maintain karta hai. Ye data safety ke liye important hota hai.

Main TCL commands hain:

- * **COMMIT** -> changes permanently save karna
- * **ROLLBACK** -> changes undo karna
- * **SAVEPOINT** -> transaction ke beech me checkpoint banana

TCL ensure karta hai ki agar koi error aaye to system previous state me wapas aa sake. Ye banking jaise systems me bahut important hota hai jaha data accuracy critical hoti hai.

Transactions me TCL ACID properties follow karne me help karta hai. Ye data integrity aur reliability maintain karta hai.

Example:

Agar bank transfer me paise deduct ho gaye but add nahi hue, to **ROLLBACK** se transaction cancel ho jayega.

Successful transaction ke baad **COMMIT** se changes save ho jate hain.

Topic 12: Database Design

Database Design ka matlab hai **database ka structure plan karna** taaki data efficiently store aur manage ho sake. Ye process system ki requirements samajhne se start hota hai. Fir entities, attributes aur relationships identify kiye jate hain. Proper design se data redundancy kam hoti hai aur performance improve hoti hai.

Database design me **conceptual, logical aur physical design phases** hote hain. Conceptual design me ER model banaya jata hai. Logical design me tables aur schema define hote hain. Physical design me storage aur indexing decide hoti hai.

Good design se data consistency aur integrity maintain hoti hai. Ye future modifications aur scalability ko easy banata hai. Poor design se data duplication, errors aur slow performance ho sakti hai.

Normalization bhi database design ka important part hai jo data ko structured aur optimized banata hai.

Designers ko user requirements aur system performance dono consider karna padta hai.

Example:

College database design me Student, Course aur Faculty tables create ki jati hain aur unke relationships define kiye jate hain.

Topic 13: ER Diagram

ER Diagram (Entity-Relationship Diagram) ek **graphical representation** hai jo database design ko visually show karta hai. Isme entities, attributes aur relationships ko diagram ke form me dikhaya jata hai. Ye conceptual design phase me use hota hai. ER diagram se database structure easily samajh aata hai. Isme different symbols use hote hain:

* Rectangle -> Entity

* Oval -> Attribute

* Diamond -> Relationship

ER diagram designer ko clear view deta hai ki data kaise connected hai. Ye redundancy avoid karne me help karta hai. Complex systems ko simple way me represent karne ke liye useful hai. Implementation se pehle errors identify karne me bhi help karta hai.

Ye communication tool bhi hai jisse developers aur users easily design discuss kar sakte hain. Proper ER diagram database design ko strong banata hai.

Example:

Student entity (RollNo, Name) aur Course entity (CourseID) ko "enrolls" relationship se connect kiya ja sakta hai ER diagram me.

Topic 14: Entities and Attributes

Entities aur Attributes ER model ke **basic building blocks** hote hain.

Entity ka matlab hai koi real-world object ya concept jise hum database me represent karte hain. Ye physical ya logical dono ho sakta hai. Example: Student, Employee, Product.

Attributes entity ki properties hoti hain jo uske baare me information deti hain. Example: Student ke attributes ho sakte hain RollNo, Name, Age.

Attributes ke types bhi hote hain jaise simple, composite (Name -> First + Last), single-valued aur multi-valued (Phone numbers). Primary key attribute uniquely har entity ko identify karta hai.

Entities aur attributes milkar database ka structure define karte hain. Ye data ko organize karne me help karte hain. Proper identification se database efficient aur accurate banta hai.

Example:

Student entity -> (RollNo, Name, Marks)

Yaha RollNo primary key hai jo har student ko uniquely identify karta hai.

Topic 15: Entity Sets

Entity Set ka matlab hai **same type ki multiple entities ka collection**. Yani ek category ke sab objects ko ek group me represent kiya jata hai. Ye database me data ko organize karne me help karta hai. Har entity set me same attributes hote hain. Ye table ke similar hota hai relational model me.

Entity set me har entity uniquely identify hoti hai using primary key. Ye logical grouping provide karta hai.

Isse data ko manage aur query karna easy ho jata hai. Database design me entity sets important role play karte hain. Ye ER model ka core concept hai.

Entity sets do types ke hote hain: strong entity set aur weak entity set. Strong entity apne aap exist kar sakti hai, jabki weak entity kisi dusri entity par depend hoti hai.

Proper entity set design se redundancy kam hoti hai aur data consistency maintain hoti hai.

Example:

Student entity set me sabhi students ka data hoga: (101, Rahul), (102, Priya).

Yaha sab students ek hi entity set ka part hain.

Topic 16: Relationships and Relationship Sets

Relationship ka matlab hai **do ya zyada entities ke beech connection**. Ye batata hai ki entities ek dusre se kaise related hain. Relationship database design me real-world interactions ko represent karta hai.

Example: Student enrolls in Course.

Relationship Set same type ke relationships ka collection hota hai. Ye entity set ki tarah hi grouping provide karta hai. Har relationship me participating entities hoti hain. Relationships ke types hote hain: one-to-one, one-to-many aur many-to-many.

Relationship ke saath attributes bhi ho sakte hain, jaise enrollment date. Ye additional information provide karta hai. Proper relationships define karne se data consistency maintain hoti hai.

Database design me relationships important hote hain kyunki ye tables ke beech connection establish karte hain. Ye queries ko efficient banata hai.

Example:

Student aur Course ke beech "enrolls" relationship ho sakta hai.

Ek student multiple courses me enroll ho sakta hai (one-to-many).

Topic 17: ER Design Issues

ER Design Issues ka matlab hai **ER model design karte waqt aane wali common problems aur decisions**. Ye issues database ko sahi tarike se design karne me important hote hain. Agar galat design ho jaye to future me redundancy aur inconsistency badh sakti hai.

Ek main issue hai **entity vs attribute decision**. Kab kisi cheez ko entity banana hai aur kab attribute, ye decide karna hota hai. Dusra issue hai **entity vs relationship**, yani kab relation alag define karna chahiye.

Binary vs n-ary relationships bhi ek issue hai jaha decide karna hota hai kitni entities include karni hain.

Aggregation ka use tab hota hai jab relationship ko ek higher-level entity ki tarah treat karna ho. Weak entities ko identify karna bhi important hai. Proper keys define karna bhi ek major concern hota hai.

Good ER design se database efficient aur flexible banta hai. Ye future changes aur queries ko easy banata hai. Poor design se duplication aur errors increase hote hain.

Example:

"Address" ko attribute rakhna ya separate entity banana ek design issue hai.

StudentCourseTeacher ko binary ya ternary relationship me represent karna bhi decision hota hai.

Unit 2

Topic 1: Introduction to Relational Model

Relational Model database ka **most widely used model** hai jisme data ko **tables (relations)** ke form me store kiya jata hai. Har table rows (tuples) aur columns (attributes) se milkar banti hai. Ye model simple aur easy to understand hota hai, isliye practical systems me zyada use hota hai.

Is model me har row ek record represent karti hai aur har column ek property. Tables ke beech relationships **keys (primary key aur foreign key)** ke through define hote hain. Ye data consistency aur integrity maintain karta hai.

Relational model ka foundation **set theory aur predicate logic** par based hai. Isme duplicate rows allowed nahi hote aur order matter nahi karta. SQL language isi model par based hai jisse data ko easily query kiya ja sakta hai.

Ye model flexible hai, jisme data ko easily insert, update aur delete kiya ja sakta hai. Normalization ka concept bhi isi model me use hota hai jisse redundancy kam hoti hai.

Modern databases jaise MySQL, Oracle, PostgreSQL sab relational model follow karte hain. Ye model scalability aur performance dono provide karta hai.

Example:

Student table: (RollNo, Name, Marks)

Course table: (CourseID, CourseName)

Foreign key ke through dono tables connect ho sakte hain.

Topic 3: Database Schema

Database Schema ka matlab hai **database ka overall structure ya blueprint**. Ye define karta hai ki database me tables kaise honge, unke columns kya honge aur relationships kaise hongie. Schema design time par define hota hai aur rarely change hota hai. Ye database ka logical view hota hai.

Schema me tables, attributes, data types aur constraints include hote hain. Ye batata hai ki data ka format kya hoga aur rules kya honge. Isse data consistency aur integrity maintain hoti hai. Schema ke basis par hi database create hota hai.

Schema aur instance me difference hota hai. Schema fixed structure hota hai, jabki instance actual data hota hai jo change hota rehta hai. Proper schema design se database efficient aur scalable banta hai.

Different types ke schema hote hain jaise physical schema, logical schema aur view schema. Ye abstraction levels ko represent karte hain.

Schema database development ka important part hai kyunki ye future operations ko affect karta hai.

Example:

Student table schema ho sakta hai: (RollNo INT, Name VARCHAR, Marks INT).

Ye structure define karta hai, actual values instance me store hoti hain.

Topic 4: Keys

Keys database me **records ko uniquely identify karne aur tables ko connect karne** ke liye use hoti hain. Ye relational model ka core concept hai. Keys data integrity maintain karti hain aur duplication avoid karti hain. Har table me kam se kam ek key hona zaroori hota hai.

Primary Key sabse important hoti hai jo har record ko uniquely identify karti hai. Isme NULL ya duplicate values allowed nahi hoti. **Foreign Key** ek table ko dusre table se connect karti hai. Ye relationship establish karti hai.

Aur bhi types hote hain jaise **Candidate Key** (possible keys), **Super Key** (extra attributes ke saath unique identifier), aur **Composite Key** (multiple attributes se bani key).

Keys ensure karti hain ki database me data consistent aur valid rahe. Ye queries ko efficient banati hain aur relationships manage karne me help karti hain.

Proper key selection database performance aur design ke liye important hota hai.

Example:

Student table me RollNo primary key ho sakta hai.

Marks table me RollNo foreign key ho sakta hai jo Student table se link kare.

Topic 5: Schema Diagram

Schema Diagram ek **visual representation** hota hai jo database ke structure ko diagram ke form me dikhata hai. Ye tables, attributes aur unke relationships ko clearly show karta hai. Isse database design ko samajhna easy ho jata hai. Ye developers aur designers ke liye important tool hai.

Schema diagram me har table ek box ke form me hota hai jisme columns (attributes) likhe hote hain.

Primary key ko usually highlight kiya jata hai. Tables ke beech connections lines ke through dikhaye jate hain, jo foreign keys ko represent karte hain.

Ye diagram database ka overview deta hai aur design errors identify karne me help karta hai. Ye documentation ke liye bhi useful hota hai. Complex database systems ko simple tarike se represent karne me madad karta hai.

Schema diagram se query writing aur database understanding dono easy ho jate hain. Ye database development ka important part hai.

Example:

Student table (RollNo, Name) aur Marks table (RollNo, Subject, Marks) ko line ke through connect karke schema diagram banaya jata hai.

Topic 6: Basic Structure of SQL Queries

SQL (Structured Query Language) ka use **database se data retrieve aur manipulate karne** ke liye hota hai. Basic SQL query ka structure simple hota hai aur mainly **SELECT, FROM, WHERE** clauses par based hota hai. Ye database ke tables se required data nikalne me help karta hai.

Sabse common structure hai:

SELECT column_name(s) FROM table_name WHERE condition;

* **SELECT** -> kaunse columns chahiye

* **FROM** -> kis table se data lena hai

* **WHERE** -> kaunsi condition apply karni hai

Iske alawa **ORDER BY** sorting ke liye, **DISTINCT** duplicate remove karne ke liye aur **LIMIT** records restrict karne ke liye use hota hai. SQL case-insensitive hota hai, matlab keywords uppercase ya lowercase dono me likh sakte hain.

SQL queries non-procedural hoti hain, yani user sirf batata hai kya chahiye, system khud decide karta hai kaise lana hai. Ye data access ko simple aur powerful banata hai.

Proper SQL structure se queries efficient aur readable banti hain. Ye database operations ka basic foundation hai.

Example:

sql

SELECT Name, Marks FROM Student WHERE Marks > 50;

Ye query un students ka data degi jinke marks 50 se zyada hain.

Topic 7: Set Operations

Set Operations SQL me **multiple queries ke results ko combine ya compare** karne ke liye use hote hain. Ye mathematical set theory par based hote hain. Iska use tab hota hai jab hume alag-alag tables ya queries ke results ko ek saath process karna ho.

Main set operations hain:

* **UNION** -> do queries ke results combine karta hai aur duplicates remove karta hai

* **UNION ALL** -> combine karta hai but duplicates remove nahi karta

* **INTERSECT** -> common records return karta hai

* **EXCEPT (MINUS)** -> pehli query ke records jo dusri me nahi hain

Set operations use karte waqt dono queries me **same number of columns aur compatible data types** hone chahiye. Ye operations data comparison aur analysis me useful hote hain.

Ye queries ko powerful banate hain aur complex data retrieval easy karte hain. Reporting aur data filtering me iska use common hai.

Example:

```
sql
```

```
SELECT Name FROM Student1
```

```
UNION
```

```
SELECT Name FROM Student2;
```

Ye dono tables ke unique student names return karega.

Topic 8: Aggregate Functions

Aggregate Functions SQL me **multiple rows ke data par calculation karke single result** return karti hain. Ye data analysis aur reporting me bahut useful hoti hain. Ye individual records par nahi balki group ya entire table par apply hoti hain.

Common aggregate functions hain:

* **COUNT()** -> total records count karta hai

* **SUM()** -> numeric values ka total nikalta hai

* **AVG()** -> average calculate karta hai

* **MAX()** -> maximum value find karta hai

* **MIN()** -> minimum value find karta hai

Ye functions mostly **SELECT statement ke saath** use hote hain. NULL values ko generally ignore kiya jata hai (except COUNT(*)). Ye functions large data ko summarize karne me help karte hain.

Aggregate functions ko GROUP BY ke saath use karke group-wise results bhi nikale ja sakte hain. Ye business reports aur analysis me important role play karte hain.

Example:

```
sql id="4x4x8d"
```

```
SELECT AVG(Marks) FROM Student;
```

Ye query sabhi students ke marks ka average return karegi.

Topic 9: GROUP BY and HAVING

GROUP BY clause ka use SQL me **rows ko group karne** ke liye hota hai based on one or more columns.

Ye mostly aggregate functions ke saath use hota hai taaki har group ka summary nikala ja sake. Isse data ko category-wise analyze karna easy ho jata hai.

HAVING clause ka use **group par condition apply karne** ke liye hota hai. Ye WHERE jaisa hota hai but difference ye hai ki WHERE individual rows par condition lagata hai, jabki HAVING grouped data par condition lagata hai.

Query ka flow hota hai:

```
SELECT -> FROM -> WHERE -> GROUP BY -> HAVING
```

GROUP BY se data organize hota hai aur HAVING se filtered results milte hain. Ye reporting aur data analysis me bahut useful hai.

Example:

```
sql id="x1l9p2"
```

```
SELECT Department, AVG(Salary)
```

```
FROM Employee
```

```
GROUP BY Department
```

```
HAVING AVG(Salary) > 30000;
```

Ye query un departments ka average salary dikhayegi jaha average 30000 se zyada hai.

Topic 10: Nested Subqueries

Nested Subquery ka matlab hai **ek SQL query ke andar dusri query likhna**. Inner query pehle execute hoti hai aur uska result outer query use karti hai. Ye complex conditions aur data filtering ke liye useful hota hai.

Subqueries SELECT, FROM ya WHERE clause ke andar likhi ja sakti hain. Most common use WHERE clause me hota hai. Ye dynamic filtering allow karta hai jaha result kisi aur query par depend karta hai. Nested queries readability aur flexibility badhati hain. Ye multiple steps ko ek hi query me combine kar deti hain. Lekin zyada complex hone par performance affect ho sakti hai.

Subqueries single value (scalar), multiple values ya table return kar sakti hain. Operators jaise IN, EXISTS, ANY ke saath use hoti hain.

Ye real-world problems solve karne me powerful technique hai.

Example:

```
sql id="s8k2f1"
```

```
SELECT Name FROM Student
```

```
WHERE Marks > (SELECT AVG(Marks) FROM Student);
```

Ye query un students ko return karegi jinke marks average se zyada hain.

Topic 11: Joins

Joins ka use SQL me **multiple tables ko combine karke data retrieve karne** ke liye hota hai. Jab data alag-alag tables me store hota hai, tab joins ke through unko connect karke meaningful result nikala jata hai. Ye relational databases ka important concept hai.

Joins tables ke beech **common column (usually key)** ke basis par kaam karte hain. Common types of joins hain:

- * **INNER JOIN** -> sirf matching records return karta hai

- * **LEFT JOIN** -> left table ke sab records + matching right records

- * **RIGHT JOIN** -> right table ke sab records + matching left records

- * **FULL JOIN** -> dono tables ke sab records

Joins se data duplication avoid hota hai aur normalization maintain rehta hai. Ye complex queries ko simple banata hai.

Proper joins use karne se performance aur data accuracy dono improve hote hain. Ye reporting aur data analysis me bahut useful hai.

Example:

```
sql id="z7p3q9"
```

```
SELECT Student.Name, Marks.Marks
```

```
FROM Student
```

```
INNER JOIN Marks
```

```
ON Student.RollNo = Marks.RollNo;
```

Ye query Student aur Marks tables ko join karke result degi.

Topic 12: Triggers

Triggers database me **automatic actions** hote hain jo kisi specific event (INSERT, UPDATE, DELETE) par execute hote hain. Ye manually run nahi hote, balki event hone par khud activate ho jate hain. Triggers data integrity aur business rules enforce karne me help karte hain.

Triggers mainly **BEFORE ya AFTER events** par run hote hain. Example: BEFORE INSERT ya AFTER UPDATE. Ye auditing, validation aur logging ke liye use hote hain.

Triggers ka use karke hum ensure kar sakte hain ki koi rule violate na ho. Jaise salary negative na ho, ya delete hone se pehle data backup ho jaye. Ye database automation ka powerful feature hai.

Lekin excessive triggers performance ko slow kar sakte hain, isliye carefully use karna chahiye.

Triggers database level par execute hote hain, isliye application se independent hote hain.

Example:

```
sql id="k3m8r2"
```

```
CREATE TRIGGER check_marks
```

```
BEFORE INSERT ON Student
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF NEW.Marks < 0 THEN
```

```
    SIGNAL SQLSTATE '45000';
```

```
END IF;
```

```
END;
```

Ye trigger negative marks insert hone se rokega.

Unit 3

Topic 1: Introduction to Normalization

Normalization ek **database design technique** hai jiska goal hai **data redundancy kam karna aur data consistency improve karna**. Jab data multiple jagah repeat hota hai to errors aur inconsistency badh jati hai. Normalization is problem ko solve karta hai by dividing tables into smaller, well-structured tables.

Is process me data ko different **normal forms (1NF, 2NF, 3NF, etc.)** me convert kiya jata hai. Har normal form ek specific rule follow karta hai. Jaise repeating groups remove karna, partial dependency hatana, aur transitive dependency eliminate karna.

Normalization se database efficient banta hai aur storage bhi optimize hoti hai. Ye update, insert aur delete anomalies ko prevent karta hai. Proper normalization se data integrity maintain hoti hai.

Lekin over-normalization se performance issue bhi ho sakta hai, isliye balance maintain karna zaroori hai.

Ye relational databases ka important concept hai aur database design ka essential part hai.

Example:

Agar ek table me Student aur Course dono data ho, to normalization ke baad alag Student aur Course tables banayi jati hain.

Topic 2: Functional Dependency

Functional Dependency (FD) ka matlab hai **ek attribute ki value dusre attribute par depend karti hai**.

Simple words me, agar ek value fix ho to dusri value automatically decide ho jati hai. Ye normalization ka base concept hai.

FD ko $A \rightarrow B$ form me likhte hain, jiska matlab hai **A determine karta hai B ko**. Yaha A determinant hai aur B dependent attribute hai. Agar A ki value same hai to B ki value bhi same honi chahiye.

Functional dependency se hum identify karte hain ki table me data kaise related hai. Ye redundancy aur anomalies detect karne me help karta hai. Iske types hote hain:

- * **Trivial FD** (B already A ka part ho)

- * **Non-trivial FD**

- * **Full Functional Dependency**

- * **Partial Dependency**

FD normalization me use hota hai taaki tables ko properly divide kiya ja sake. Ye database design ko accurate aur efficient banata hai.

Example:

RollNo $\rightarrow$ Name

Agar RollNo same hai to Name bhi same hoga.

Topic 3: Lossless Decomposition

Lossless Decomposition ka matlab hai **ek table ko multiple tables me todna bina data loss ke**. Jab hum normalization karte hain, tab tables ko split kiya jata hai. Lekin split aisa hona chahiye ki original table ka data wapas reconstruct kiya ja sake.

Agar decomposition ke baad tables ko join karne par **same original data mile**, to use lossless decomposition kehte hain. Agar data missing ya extra ho jaye to wo lossy decomposition hota hai, jo galat hai.

Iska main goal hai **data integrity maintain karna**. Ye ensure karta hai ki decomposition ke baad bhi information complete aur accurate rahe.

Lossless decomposition ke liye common attribute (key) hona zaroori hota hai jisse tables properly join ho sake. Ye normalization ka important part hai.

Ye database ko structured banata hai bina information lose kiye.

Example:

Original table: (RollNo, Name, Course)

Split into:

Student (RollNo, Name)

Enrollment (RollNo, Course)

Join karne par original data wapas mil jata hai -> Lossless

Topic 4: First Normal Form (1NF)

First Normal Form (1NF) normalization ka **first step** hai jisme table ko **atomic values (single values)** me convert kiya jata hai. Matlab har cell me ek hi value honi chahiye, multiple values allowed nahi hain.

Repeating groups ya multivalued attributes remove kiye jate hain.

1NF ka main rule hai: **no repeating columns, no multiple values in a single field**. Har row uniquely identify honi chahiye (primary key hona chahiye). Ye table ko structured aur simple banata hai.

1NF se data redundancy thodi kam hoti hai aur data handling easy ho jata hai. Ye further normalization (2NF, 3NF) ke liye base create karta hai. Agar table 1NF me nahi hai to aage ke steps apply nahi kar sakte. Is process me large table ko multiple rows me convert kiya jata hai jaha repeating data alag-alag rows me store hota hai.

Example:

Before 1NF:

Student (RollNo, Subjects = Math, Science)

After 1NF:

(101, Math)

(101, Science)

Ab har field me single value hai -> 1NF satisfied

Topic 5: Second Normal Form (2NF)

Second Normal Form (2NF) ka goal hai **partial dependency ko remove karna**. Ye tab apply hota hai jab table already **1NF me ho**. 2NF ensure karta hai ki har non-key attribute **poori primary key par depend kare**, sirf uske kisi part par nahi.

Partial dependency tab hoti hai jab composite primary key ka ek part kisi attribute ko determine karta hai.

Isse redundancy aur inconsistency aati hai. 2NF me aise attributes ko alag tables me shift kiya jata hai.

Ye database ko aur structured banata hai aur duplication kam karta hai. Isse update anomalies reduce hoti hain.

2NF mostly tab relevant hota hai jab **composite key (multiple columns ki key)** use ho rahi ho. Agar single primary key hai to table automatically 2NF me hoti hai.

Is process me tables ko logically divide kiya jata hai taaki har attribute correct dependency follow kare.

Example:

Table: (RollNo, CourseID, StudentName, CourseName)

Primary Key: (RollNo, CourseID)

Problem:

StudentName sirf RollNo par depend karta hai

CourseName sirf CourseID par depend karta hai

Solution (2NF):

Student (RollNo, StudentName)

Course (CourseID, CourseName)

Enrollment (RollNo, CourseID)

Topic 6: Third Normal Form (3NF)

Third Normal Form (3NF) ka goal hai **transitive dependency ko remove karna**. Ye tab apply hota hai jab table already **2NF me ho**. 3NF ensure karta hai ki non-key attributes **sirf primary key par depend kare**, kisi dusre non-key attribute par nahi.

Transitive dependency tab hoti hai jab $A \rightarrow B$ aur $B \rightarrow C$ ho, to indirectly $A \rightarrow C$ ho jata hai. Ye redundancy create karta hai. 3NF me aise dependencies ko alag tables me divide kiya jata hai.

3NF se database aur clean aur structured ban jata hai. Ye update, insert aur delete anomalies ko reduce karta hai. Data consistency improve hoti hai.

Is form me har table me sirf relevant data hi hota hai. Ye normalization ka important step hai aur practical databases me widely use hota hai.

Example:

Table: (RollNo, DeptID, DeptName)

Primary Key: RollNo

Problem:

DeptName depends on DeptID, not directly on RollNo

Solution (3NF):

Student (RollNo, DeptID)

Department (DeptID, DeptName)

Ab transitive dependency remove ho gayi

Topic 7: Dependency Preservation

Dependency Preservation ka matlab hai **decomposition ke baad bhi saari functional dependencies ko maintain rakhna**. Jab hum normalization ke liye tables ko split karte hain, to ye ensure karna zaroori hota hai ki original dependencies lose na ho.

Agar decomposition ke baad dependencies check karne ke liye **tables ko join na karna pade**, to use dependency preserving decomposition kehte hain. Ye database performance ke liye important hai.

Iska main goal hai **data consistency aur integrity maintain karna**. Agar dependencies preserve nahi hoti to constraints enforce karna difficult ho jata hai.

Normalization me hume do cheezein ensure karni hoti hain:

1. Lossless decomposition
2. Dependency preservation

Kabhi-kabhi 3NF dependency preserve karta hai jabki BCNF me ye possible nahi hota. Isliye practical systems me 3NF prefer kiya jata hai.

Example:

Original FD: $A \rightarrow B$, $B \rightarrow C$

Decompose into tables jaha $A \rightarrow B$ aur $B \rightarrow C$ easily check ho jaye bina join ke $\rightarrow$ Dependency preserved

Topic 8: BCNF (Boyce-Codd Normal Form)

BCNF ek **advanced normal form** hai jo 3NF se bhi stricter hota hai. Iska rule hai: **har functional dependency $A \rightarrow B$ me A ek super key hona chahiye**. Matlab jo attribute kisi dusre attribute ko determine kar raha hai, wo unique identifier hona chahiye.

Kabhi-kabhi table 3NF me hoti hai but BCNF me nahi hoti. Aise cases me redundancy aur anomalies abhi bhi possible hote hain. BCNF in issues ko completely remove karta hai.

BCNF ka main goal hai **strong data consistency aur redundancy elimination**. Lekin BCNF apply karne par dependency preservation kabhi lose ho sakti hai. Isliye practical use me kabhi 3NF prefer kiya jata hai. BCNF achieve karne ke liye table ko further decompose kiya jata hai. Ye ensure karta hai ki har dependency properly key par based ho.

Ye normalization ka higher level hai aur complex database design me use hota hai.

Example:

Table: (Student, Course, Instructor)

FD: Instructor $\rightarrow$ Course

Instructor key nahi hai $\rightarrow$ BCNF violation

Solution:

Instructor (Instructor, Course)

Enrollment (Student, Instructor)

Topic 9: Higher Normal Forms

Higher Normal Forms ka matlab hai **3NF aur BCNF ke baad ke advanced normalization levels**. Inka use tab hota hai jab database me complex dependencies ho jise lower normal forms handle nahi kar pate. Ye database ko aur refine aur optimized banate hain.

Main higher normal forms hain: **4NF (Fourth Normal Form) aur 5NF (Fifth Normal Form)**. Ye multivalued aur join dependencies ko handle karte hain. Inka goal hai unnecessary data duplication aur complex anomalies ko remove karna.

Higher normal forms ka use mostly complex systems me hota hai jaha data relationships zyada complicated hote hain. Ye theoretical importance zyada rakhte hain, practical use limited hota hai.

Ye ensure karte hain ki database me har dependency properly managed ho. Lekin over-normalization se performance issue ho sakta hai, isliye carefully apply kiya jata hai.

Inka main focus hai **data integrity ko maximum level tak maintain karna**.

Example:

Agar ek table me multiple independent multi-valued attributes ho, to unhe alag tables me split karke higher normal form achieve kiya jata hai.

Topic 10: Multivalued Dependency

Multivalued Dependency (MVD) tab hoti hai jab **ek attribute ki value ke saath kisi dusre attribute ki multiple independent values exist karti hain**. Isko $A \twoheadrightarrow B$ form me likhte hain. Matlab ek A ke liye multiple B values ho sakti hain aur wo kisi aur attribute se independent hoti hain.

MVD me problem tab aati hai jab ek hi table me multiple independent multivalued attributes store hote hain. Isse redundancy aur anomalies create hoti hain. Isko solve karne ke liye table ko split kiya jata hai.

Ye concept **4NF (Fourth Normal Form)** ka base hai. Agar table me multivalued dependency ho aur wo properly handle na ho to database inefficient ho jata hai.

MVD me data duplication avoid karne ke liye decomposition kiya jata hai jisse independent data alag tables me store ho.

Ye concept complex database design me use hota hai.

Example:

Table: (Student, Hobby, Language)

Ek student ke multiple hobbies aur multiple languages ho sakte hain

MVD: Student -> Hobby

MVD: Student -> Language

Solution:

StudentHobby (Student, Hobby)

StudentLanguage (Student, Language)

Topic 11: Fourth Normal Form (4NF)

Fourth Normal Form (4NF) ka goal hai **multivalued dependencies ko remove karna**. Ye tab apply hota hai jab table already BCNF me ho. 4NF ensure karta hai ki **table me koi non-trivial multivalued dependency na ho**.

Agar ek table me ek hi key ke liye multiple independent attributes ho (jaise hobbies aur languages), to redundancy create hoti hai. 4NF me aise attributes ko alag-alag tables me divide kiya jata hai.

4NF ka main focus hai **data duplication ko completely eliminate karna** jo multivalued dependencies ki wajah se hoti hai. Ye database ko clean aur efficient banata hai.

Ye higher normalization level hai aur mostly complex databases me use hota hai. Practical systems me kabhi-kabhi 3NF ya BCNF hi sufficient hota hai.

4NF apply karne se queries thodi complex ho sakti hain, lekin data integrity improve hoti hai.

Example:

Table: (Student, Hobby, Language)

Problem:

Student -> Hobby

Student -> Language

Solution (4NF):

StudentHobby (Student, Hobby)

StudentLanguage (Student, Language)

Unit 4

Topic 1: Transaction Concept

Transaction database me **operations ka ek group** hota hai jo ek logical unit ke tarah execute hota hai. Ye multiple steps ka combination hota hai, jaise insert, update ya delete. Transaction ka main rule hai: **ya to poora complete hoga ya bilkul nahi hoga**. Isse database consistency maintain hoti hai.

Transaction ka use real-life applications me hota hai jaha data accuracy important hoti hai. Ye system failures ya errors ke time data ko safe rakhta hai. Agar transaction fail ho jaye to system previous state me wapas aa jata hai.

Transaction database ko reliable banata hai. Ye ACID properties follow karta hai jo data integrity ensure karti hain. Multiple users ek hi time par transactions perform kar sakte hain.

Transaction management se data conflicts aur inconsistencies avoid hoti hain. Ye database systems ka core concept hai.

Example:

Bank transfer me paise ek account se deduct hote hain aur dusre me add hote hain. Agar koi step fail ho jaye to pura transaction cancel ho jata hai.

Topic 2: Transaction Properties (ACID)

ACID properties transaction ki **reliability aur correctness ensure** karti hain. Ye 4 rules hote hain:

Atomicity, Consistency, Isolation, Durability. Har transaction ko in properties follow karna hota hai.

Atomicity ka matlab hai transaction ya to poora execute hoga ya bilkul nahi. Partial execution allowed nahi hota.

Consistency ensure karta hai ki transaction ke baad database valid state me rahe.

Isolation ka matlab hai ek transaction dusre par effect nahi dale jab tak wo complete na ho.

Durability ensure karta hai ki ek baar transaction commit ho jaye to changes permanently save ho jaye.

Ye properties system failures ke time data ko safe rakhti hain. Ye database ko reliable aur error-free banati hain.

ACID rules banking, e-commerce jaise systems me bahut important hote hain jaha data accuracy critical hoti hai.

Example:

Bank transfer me agar paise deduct ho gaye but add nahi hue, to atomicity ensure karega ki pura transaction rollback ho jaye.

Commit hone ke baad data permanent save ho jata hai (durability).

Topic 3: Transaction Management using SQL

Transaction Management using SQL ka matlab hai **SQL commands ka use karke transactions ko control karna**. Ye ensure karta hai ki database operations safe aur consistent rahe. Transactions ko manage karne ke liye mainly TCL commands use hote hain jaise **COMMIT, ROLLBACK aur SAVEPOINT**. Transaction normally **BEGIN ya START TRANSACTION** se start hota hai. Uske baad multiple SQL operations perform hote hain. Agar sab operations successfully complete ho jaye to **COMMIT** use karke changes permanently save kar diye jate hain. Agar koi error aaye to **ROLLBACK** use karke sab changes cancel kar diye jate hain.

SAVEPOINT transaction ke beech me checkpoint create karta hai jaha tak rollback kiya ja sakta hai. Ye large transactions me useful hota hai jaha partial undo karna ho.

SQL transaction management ACID properties follow karne me help karta hai. Ye data consistency aur integrity maintain karta hai. Ye multi-user environment me conflicts avoid karta hai.

Proper transaction management se system reliable aur fault-tolerant banta hai.

Example:

```
sql id="t1m3k9"
```

```
START TRANSACTION;
```

```
UPDATE Account SET Balance = Balance - 500 WHERE AccNo = 1;
```

```
UPDATE Account SET Balance = Balance + 500 WHERE AccNo = 2;
```

```
COMMIT;
```

Agar koi step fail ho jaye to ROLLBACK use kiya ja sakta hai.

Topic 4: Transaction Log

Transaction Log ek **special file** hota hai jisme database ke sabhi transactions ka **record maintain kiya jata hai**. Ye har operation jaise INSERT, UPDATE, DELETE ka detail store karta hai. Iska main purpose hai **recovery aur data safety**.

Jab bhi koi transaction perform hota hai, uska record log me save hota hai before actual change database me apply ho. Isse system crash hone par data recover kiya ja sakta hai. Transaction log me **old value aur new value** dono store hoti hain.

Transaction log recovery techniques jaise **redo aur undo** me use hota hai. Redo committed transactions ko wapas apply karta hai aur undo uncommitted transactions ko cancel karta hai.

Ye database reliability aur durability maintain karta hai. ACID property (especially durability) me iska important role hota hai.

Large systems me transaction log continuously update hota rehta hai. Proper log management performance ke liye bhi important hai.

Example:

Agar system crash ho jaye transaction ke beech me, to log ke basis par database ko previous consistent state me restore kiya jata hai.

Topic 5: Concurrency Control

Concurrency Control ka matlab hai **multiple users ke simultaneous transactions ko manage karna** taaki data inconsistency na ho. Jab ek hi data par ek time par multiple operations hote hain, to conflicts ho sakte hain. Concurrency control in problems ko prevent karta hai.

Iska main goal hai **database consistency maintain karna**. Ye ensure karta hai ki transactions isolated tareeke se execute ho. Common problems hain: dirty read, lost update aur uncommitted data access. Concurrency control techniques jaise **locking, timestamping aur validation** use hoti hain. Ye transactions ke order ko control karti hain.

Ye ACID property (especially isolation) ko maintain karta hai. Multi-user systems me ye bahut important hota hai.

Proper concurrency control se system reliable aur error-free banta hai.

Example:

Do users ek hi bank account ko same time par update kar rahe ho, to concurrency control ensure karega ki final balance correct ho.

Topic 6: Locking Methods

Locking Methods concurrency control ka ek **important technique** hai jisme database data items par **locks lagakar access control kiya jata hai**. Jab ek transaction kisi data ko use karta hai, to us par lock lag jata hai taaki dusre transactions usko simultaneously modify na kar sake.

Locking ka main goal hai **data inconsistency aur conflicts ko prevent karna**. Isse ensure hota hai ki ek time par sirf authorized transaction hi data access kare. Locking se problems jaise lost update aur dirty read avoid hote hain.

Locking do tarah ka hota hai:

- * **Shared Lock (Read Lock)** -> multiple transactions read kar sakte hain

- * **Exclusive Lock (Write Lock)** -> sirf ek transaction write kar sakta hai

Locking mechanism transactions ko control karta hai aur data integrity maintain karta hai. Lekin improper locking se **deadlock** bhi ho sakta hai.

Ye multi-user database systems me bahut important hota hai jaha concurrent access hota hai.

Example:

Agar ek transaction kisi account balance ko update kar raha hai, to dusra transaction usko modify nahi kar sakta jab tak pehla complete na ho.

Topic 7: Types of Locks

Database me locks ka use **data access control karne** ke liye hota hai. Types of locks batate hain ki kaun sa operation allowed hai (read ya write). Ye concurrency control ka important part hai.

Main types of locks hain:

- * **Shared Lock (S Lock)** -> sirf read allow karta hai, multiple transactions ek saath read kar sakte hain

- * **Exclusive Lock (X Lock)** -> read + write dono allow karta hai, lekin sirf ek transaction ke liye

- * **Update Lock** -> read ke baad write me convert ho sakta hai

- * **Intent Locks** -> hierarchical locking me use hote hain

Shared lock data ko modify nahi hone deta jab tak read ho raha hai. Exclusive lock ensure karta hai ki data safe tarike se update ho.

Proper lock selection se system efficient aur consistent rehta hai. Galat locking se deadlock ya performance issues ho sakte hain.

Locks database operations ko safe aur controlled banate hain.

Example:

Agar ek user data read kar raha hai (S lock), to dusra bhi read kar sakta hai.

Lekin agar koi update kar raha hai (X lock), to dusra user wait karega.

Topic 8: Two Phase Locking (2PL)

Two Phase Locking (2PL) ek **concurrency control protocol** hai jo ensure karta hai ki transactions **serializable order** me execute ho. Iska main goal hai data consistency maintain karna. 2PL me transaction do phases follow karta hai: **Growing Phase aur Shrinking Phase**.

Growing Phase me transaction locks acquire karta hai (read/write locks), lekin release nahi karta.

Shrinking Phase me transaction locks release karta hai, lekin naye locks acquire nahi kar sakta.

Is rule ki wajah se transaction ka execution controlled hota hai aur conflicts avoid hote hain. 2PL serializability guarantee karta hai, jo correct execution ka important condition hai.

Iske types bhi hote hain jaise **Strict 2PL** jisme locks commit tak release nahi hote, jo recovery ko easy banata hai.

2PL se concurrency control strong hota hai lekin deadlocks ka risk bhi badh sakta hai.

Ye database systems me widely used technique hai.

Example:

Ek transaction pehle required locks le leta hai (growing phase), fir operations complete hone ke baad locks release karta hai (shrinking phase).

Topic 9: Serializability

Serializability ka matlab hai **concurrent transactions ka result aisa ho jaise wo ek ke baad ek (serial order) me execute hue ho**. Ye concurrency control ka important concept hai. Iska goal hai **correct aur consistent output ensure karna**.

Jab multiple transactions parallel execute hote hain, to conflicts ho sakte hain. Serializability ensure karta hai ki final result same ho jo serial execution me hota. Isse data integrity maintain hoti hai.

Do main types hote hain:

* **Conflict Serializability** -> operations ke conflicts ke basis par check hota hai

* **View Serializability** -> final result ke basis par check hota hai

Serializability schedule (execution order) ko analyze karke decide kiya jata hai. Ye dependency graph ke through check kiya jata hai.

Ye ACID property (isolation) ko maintain karne me help karta hai. Proper serializable schedule se database reliable rehta hai.

Example:

Agar T1 aur T2 parallel run ho rahe hain, to serializability ensure karega ki final result T1->T2 ya T2->T1 jaisa hi ho.

Topic 10: Deadlocks

Deadlock ek situation hai jaha **do ya zyada transactions ek dusre ka wait karte rehte hain** aur koi bhi aage execute nahi ho pata. Ye tab hota hai jab resources (locks) circular wait condition me phas jate hain. Example: T1 resource A hold karke B ka wait kar raha hai, aur T2 resource B hold karke A ka wait kar raha hai. Dono wait karte rahenge -> deadlock.

Deadlock ke 4 conditions hote hain:

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

Deadlock system performance ko affect karta hai aur transactions block ho jate hain. Isko handle karne ke liye techniques use hoti hain jaise **deadlock prevention, avoidance aur detection**.

Detection ke liye wait-for graph use hota hai aur recovery me ek transaction ko rollback kiya jata hai.

Deadlock avoid karne ke liye proper locking strategy zaroori hoti hai.

Example:

Transaction T1 aur T2 ek dusre ke locks ka wait karte hain aur system freeze ho jata hai -> deadlock.

Topic 11: Concurrency Control Methods

Concurrency Control Methods wo techniques hain jo **multiple transactions ko safely manage karne** ke liye use hoti hain. Inka goal hota hai **data consistency aur isolation maintain karna** jab multiple users ek hi time par database access kar rahe ho.

Main methods hain:

1. **Lock-based Protocols** -> locks (shared/exclusive) use karke access control
2. **Timestamp-based Protocols** -> har transaction ko timestamp assign hota hai aur uske basis par execution order decide hota hai
3. **Validation-based Protocols** -> transaction complete hone par check hota hai ki conflict hua ya nahi
4. **Multiversion Concurrency Control (MVCC)** -> multiple versions of data maintain hote hain taaki read-write conflicts avoid ho

Har method ka apna advantage aur limitation hota hai. Locking simple hai but deadlock ka risk hota hai. Timestamp deadlock avoid karta hai but complex hota hai.

Ye methods ACID properties maintain karne me help karte hain. Proper method selection system performance aur consistency dono improve karta hai.

Example:

MVCC me ek user data read karta hai jabki dusra update karta hai, dono conflict ke bina kaam kar sakte hain.

Unit 5

Topic 1: File Organization

File Organization ka matlab hai **data ko disk par kaise store aur arrange kiya jata hai**. Ye database performance ko directly affect karta hai. Proper file organization se data access fast aur efficient hota hai. Different types ke file organization hote hain:

- * **Sequential File Organization** -> data order me store hota hai
- * **Indexed File Organization** -> index ke through fast access milta hai
- * **Direct (Hashed) File Organization** -> hash function se direct location milti hai

Sequential me search slow hota hai but simple hota hai. Indexed me fast search hota hai but extra storage lagta hai. Hashed method me direct access milta hai but collisions ka issue hota hai.

File organization ka selection application requirement par depend karta hai. Ye data retrieval, insertion aur deletion operations ko affect karta hai.

Proper organization se database performance improve hoti hai aur time complexity kam hoti hai.

Example:

Library system me books ko sorted order me rakhna sequential organization hai.

Index use karke quickly book find karna indexed organization ka example hai.

Topic 2: Types of Indexes

Indexes database me **data access ko fast banane ke liye use hote hain**. Ye book ke index ki tarah kaam karte hain jaha se directly required data mil jata hai. Index create karne se search time kam ho jata hai, lekin extra storage lagti hai.

Main types of indexes hain:

- * **Primary Index** -> primary key par based hota hai, unique hota hai
- * **Secondary Index** -> non-key attributes par based hota hai
- * **Clustering Index** -> data physically sorted hota hai
- * **Non-Clustering Index** -> data sorted nahi hota, sirf pointers hote hain
- * **Dense Index** -> har record ka index entry hota hai
- * **Sparse Index** -> kuch selected records ka index hota hai

Indexes read operations fast karte hain lekin insert/update thoda slow ho sakta hai. Isliye balance maintain karna zaroori hai.

Proper indexing se query performance improve hoti hai aur large databases me efficiency badh jati hai.

Example:

Student table me RollNo par index create karne se student record instantly find ho jata hai.

Topic 3: Query Processing

Query Processing ka matlab hai **SQL query ko execute karne ka process** jisme database system decide karta hai ki data ko kaise retrieve kiya jaye. Jab user query likhta hai, to system usko directly run nahi karta, balki multiple steps me process karta hai.

Steps of query processing:

1. **Parsing** -> query syntax check hoti hai
 2. **Translation** -> query ko internal form me convert kiya jata hai
 3. **Optimization** -> best execution plan choose kiya jata hai
 4. **Execution** -> actual data retrieve kiya jata hai
- Query optimizer important role play karta hai kyunki ye fastest way choose karta hai data access karne ka. Ye indexes aur statistics use karta hai.

Efficient query processing se performance improve hoti hai aur response time kam hota hai. Ye large databases me bahut important hai.

Poor query processing se system slow ho sakta hai.

Example:

Agar SELECT query me index available hai, to system full table scan ki jagah direct index use karega -> fast result.

Topic 4: Query Cost Measures

Query Cost Measures ka matlab hai **query execute karne me kitna cost (time aur resources) lagta hai**. Database system different execution plans compare karta hai aur sabse efficient plan choose karta hai. Ye performance optimization ka important part hai.

Cost mainly **disk I/O operations** par depend karta hai kyunki disk access slow hota hai. CPU time aur memory usage bhi consider kiya jata hai. Query optimizer in factors ko analyze karke best plan select karta hai.

Common cost factors:

- * Disk access (block read/write)
- * CPU processing time
- * Memory usage
- * Number of tuples processed

Index use karne se cost kam hota hai kyunki direct access milta hai. Full table scan me cost zyada hota hai.

Query cost measure se system fast response provide karta hai aur resources efficiently use hote hain. Ye large databases me critical role play karta hai.

Example:

Agar table me index hai to query cost low hoga, warna pura table scan karna padega -> high cost.

Topic 5: Selection Operations

Selection Operation ka matlab hai **table se specific rows (records) ko filter karna based on condition**.

Ye relational algebra ka basic operation hai aur SQL me WHERE clause ke through use hota hai.

Isme hum condition apply karte hain jaise =, >, <, BETWEEN, LIKE etc. Selection sirf un rows ko return karta hai jo condition satisfy karti hain. Ye data retrieval ko efficient banata hai.

Selection operation performance par depend karta hai ki data kaise access ho raha hai. Agar index available ho to fast retrieval hota hai, warna full table scan hota hai.

Selection operations query processing ka important part hain aur almost har query me use hote hain. Ye unnecessary data ko filter karke relevant data provide karta hai.

Multiple conditions AND, OR operators ke saath combine ki ja sakti hain.

Example:

sql

```
SELECT * FROM Student WHERE Marks > 60;
```

Ye query sirf un students ko show karegi jinke marks 60 se zyada hain.

Topic 6: Join Operations

Join Operations ka use **multiple tables ke data ko combine karne** ke liye hota hai. Jab data alag tables me store hota hai, to join ke through unhe ek saath laake meaningful result nikala jata hai. Ye relational algebra aur SQL dono me important concept hai.

Join me tables ke beech **common attribute (key)** use hota hai. Ye matching records ko combine karta hai.

Different types ke joins hote hain:

- * **Inner Join** -> sirf matching records

- * **Left Join** -> left table ke sab records + matching right

- * **Right Join** -> right table ke sab records + matching left

- * **Full Join** -> dono tables ke sab records

Join operations data redundancy avoid karte hain aur normalization maintain rakhte hain. Ye complex queries ko simple banata hai.

Join performance indexes aur query optimization par depend karti hai. Large tables me efficient join important hota hai.

Example:

sql id="8l2n5x"

```
SELECT Student.Name, Marks.Marks
```

```
FROM Student
```

```
JOIN Marks ON Student.RollNo = Marks.RollNo;
```

Ye query dono tables ka combined result degi.

Topic 7: Set Operations

Set Operations ka use **multiple query results ko combine ya compare karne** ke liye hota hai. Ye relational algebra aur SQL dono me use hote hain. Ye operations mathematical set theory par based hote hain.

Main set operations hain:

- * **UNION** -> do results ko combine karta hai aur duplicates remove karta hai

- * **UNION ALL** -> combine karta hai but duplicates remove nahi karta

- * **INTERSECT** -> common records return karta hai

- * **EXCEPT (MINUS)** -> pehli query ke records jo dusri me nahi hain

Set operations use karte waqt dono queries me **same number of columns aur compatible data types** hone chahiye. Ye data comparison aur filtering me useful hote hain.

Ye operations reporting aur analysis me kaafi helpful hote hain. Ye complex queries ko simplify karte hain.

Example:

sql id="7d3k9q"

```
SELECT Name FROM Student1
```

```
UNION
```

```
SELECT Name FROM Student2;
```

Ye dono tables ke unique names return karega.

Topic 8: Aggregate Operations

Aggregate Operations ka use **multiple rows ke data par calculation karke single summarized result** nikalne ke liye hota hai. Ye relational algebra aur SQL dono me important concept hai. Ye large data ko analyze karne me help karta hai.

Common aggregate operations hain:

- * **COUNT** -> total records count karta hai
- * **SUM** -> total value calculate karta hai
- * **AVG** -> average nikalta hai
- * **MAX** -> maximum value find karta hai
- * **MIN** -> minimum value find karta hai

Ye operations mostly **GROUP BY** ke saath use hote hain taaki group-wise results mile. Ye reporting, analytics aur decision making me important role play karte hain.

Aggregate operations NULL values ko generally ignore karte hain (except COUNT(*)). Ye data ko summarize karke easy understanding provide karte hain.

Performance indexes aur query optimization par depend karti hai.

Example:

```
sql id="5m8p2r"
```

```
SELECT AVG(Marks) FROM Student;
```